

app\temperature_conversion.py

```
1  # -*- coding: utf-8 -*-
2  """
3  @file      temperature_conversion.py
4  @brief     Module de conversion résistance/température via interpolation linéaire.
5  @details   Ce module fournit les fonctions de conversion bidirectionnelle
6             entre résistance mesurée et température pour différents types
7             de capteurs. Utilise l'interpolation linéaire sur les tables
8             de calibration et supporte la conversion inverse pour la
9             simulation de résistance.
10 @author    Léo Mendes
11 @project   2510_RegulationsChauffage
12 @Mandant   Oblo_solution
13 @date      2025-09-18
14 @version   1.0.0
15 @copyright Copyright (c) 2025
16 """
17
18 # Import des tables de capteurs depuis le module de profils
19 from app.sensor_profiles import SENSOR_TABLES
20
21
22 # -----
23 # Fonctions de conversion de base
24 # -----
25
26 def resistance_to_temperature(resistance, table):
27     """
28     @brief   Convertit une résistance mesurée en température via interpolation linéaire.
29     @details Algorithme d'interpolation linéaire qui recherche l'intervalle
30             contenant la résistance mesurée et calcule la température
31             correspondante. Gère les cas limites hors plage de mesure.
32
33     @param resistance Résistance mesurée [Ohms].
34     @param table      Liste de tuples (résistance, température) pour interpolation.
35
36     @return           Température interpolée [°C], ou valeurs limites si hors plage.
37     """
38     # Parcours des paires de points consécutifs dans la table de conversion
39     for i in range(len(table) - 1):
40         # Extraction des coordonnées du premier point de l'intervalle
41         r1, t1 = table[i]
42         # Extraction des coordonnées du second point de l'intervalle
43         r2, t2 = table[i + 1]
44         # Vérification si la résistance mesurée est dans l'intervalle courant
45         if r1 <= resistance <= r2:
46             # Application de la formule d'interpolation linéaire
47             temperature = t1 + (resistance - r1) / (r2 - r1) * (t2 - t1)
48             # Retour de la température interpolée
49             return temperature
50
51     # Gestion des cas hors limites de la table de conversion
52     r_min, t_min = table[0]
53     # Extraction des valeurs maximales de la table
54     r_max, t_max = table[-1]
55     # Retour de la température minimale si résistance trop faible
56     if resistance < r_min:
57         return t_min
58     # Retour de la température maximale si résistance trop élevée
59     if resistance > r_max:
60         return t_max
61     # Retour de None si aucune condition n'est satisfaite (cas d'erreur)
62     return None
63
64 def resistance_to_temperature_dynamic(resistance, sensor):
65     """
66     @brief   Convertit une résistance mesurée en température pour un capteur donné.
67     @details Fonction dynamique qui sélectionne automatiquement la table de
68             conversion appropriée selon le type de capteur spécifié.
69             Effectue la validation du capteur avant conversion.
70
71     @param resistance Résistance mesurée [Ohms].
72     @param sensor      Nom du type de capteur à utiliser.
73
74     @return           Température interpolée [°C] ou None si hors limites.
75
76     @exception        ValueError si le capteur n'est pas reconnu.
77     """
78     # Vérification de l'existence du capteur dans les tables disponibles
79     if sensor not in SENSOR_TABLES:
80         # Levée d'exception avec message explicite si capteur introuvable
81         raise ValueError(f"Capteur {sensor} non trouvé dans les données.")
82
83     # Récupération de la table de conversion spécifique au capteur
84     table = SENSOR_TABLES[sensor]
```

```
85 # Parcours des paires de points consécutifs dans la table
86 for i in range(len(table) - 1):
87     # Extraction température et résistance du premier point (format inversé)
88     t1, r1 = table[i]
89     # Extraction température et résistance du second point
90     t2, r2 = table[i + 1]
91     # Vérification si la résistance mesurée est dans l'intervalle
92     if r1 <= resistance <= r2:
93         # Application de l'interpolation linéaire pour calculer la température
94         temperature = t1 + (resistance - r1) / (r2 - r1) * (t2 - t1)
95         # Retour de la température calculée
96         return temperature
97
98 # Gestion des valeurs minimales et maximales de la table
99 t_min, r_min = table[0]
100 # Extraction des valeurs maximales
101 t_max, r_max = table[-1]
102 # Retour de la température minimale si résistance trop faible
103 if resistance < r_min:
104     return t_min
105 # Retour de la température maximale si résistance trop élevée
106 if resistance > r_max:
107     return t_max
108 # Retour de None si aucune condition n'est remplie
109 return None
110
111 def temperature_to_resistance_dynamic(temperature, sensor):
112     """
113     @brief Convertit une température en résistance pour un capteur donné (conversion inverse).
114     @details Fonction de conversion inverse utilisée pour la simulation de résistance.
115             Effectue l'interpolation linéaire sur l'axe température pour obtenir
116             la résistance correspondante selon le type de capteur.
117
118     @param temperature Température d'entrée [°C].
119     @param sensor       Nom du type de capteur à utiliser.
120
121     @return             Résistance interpolée [Ohms] ou None si hors limites.
122
123     @exception          ValueError si le capteur n'est pas reconnu.
124     """
125     # Vérification de l'existence du capteur dans les tables de conversion
126     if sensor not in SENSOR_TABLES:
127         # Levée d'exception avec message d'erreur explicite
128         raise ValueError(f"Capteur {sensor} non trouvé dans les données.")
129
130     # Récupération de la table de conversion pour le capteur spécifié
131     table = SENSOR_TABLES[sensor]
132
133     # Parcours des paires de points consécutifs pour la recherche d'intervalle
134     for i in range(len(table) - 1):
135         # Extraction des coordonnées du premier point (température, résistance)
136         t1, r1 = table[i]
137         # Extraction des coordonnées du second point
138         t2, r2 = table[i + 1]
139
140         # Vérification si la température est contenue dans l'intervalle courant
141         if t1 <= temperature <= t2:
142             # Application de l'interpolation linéaire inverse (température -> résistance)
143             resistance = r1 + (temperature - t1) / (t2 - t1) * (r2 - r1)
144             # Retour de la résistance calculée
145             return resistance
146
147     # Gestion des cas hors limites de la table de conversion
148     t_min, r_min = table[0]
149     # Extraction des valeurs maximales de température et résistance
150     t_max, r_max = table[-1]
151
152     # Retour de la résistance minimale si température trop basse
153     if temperature < t_min:
154         return r_min
155     # Retour de la résistance maximale si température trop élevée
156     if temperature > t_max:
157         return r_max
158
159     # Retour de None si aucune condition n'est satisfaite
160     return None
161
162 def convert_temperature_to_resistance(temperature, probe_type):
163     """
164     @brief Fonction de conversion principale pour la simulation de résistance.
165     @details Fonction de haut niveau qui supporte différents types de sondes
166             avec leurs caractéristiques spécifiques. Inclut la normalisation
167             des noms de sondes et la gestion des variantes de nomenclature.
168
169     @param temperature Température simulée [°C].
170     @param probe_type   Type de sonde (formats multiples supportés).
```

```

172     @return          Résistance simulée [Ohms] ou None si échec.
173     """
174     # Affichage d'information sur la conversion en cours
175     print(f"[CONV] Conversion T->R: {temperature:.2f}°C -> ? ohms (sonde: {probe_type})")
176
177     # Dictionnaire de normalisation des noms de sondes pour compatibilité
178     probe_mapping = {
179         "PT1000": "PT1000",
180         "Ni1000_TK5000": "Ni1000 TK5000",
181         "Ni1000_TK5000": "Ni1000 TK5000",
182         "NTC_10k": "NTC 10k 3977",
183         "NTC_10k_3977": "NTC 10k 3977",
184         "NTC 10k": "NTC 10k 3977"
185     }
186
187     # Recherche du nom normalisé de la sonde dans le mapping
188     normalized_probe = probe_mapping.get(probe_type, probe_type)
189
190     try:
191         # Tentative de conversion avec le nom normalisé
192         resistance = temperature_to_resistance_dynamic(temperature, normalized_probe)
193
194         # Vérification de la validité du résultat de conversion
195         if resistance is None:
196             # Affichage d'information si température hors plage supportée
197             print(f"[CONV] Température {temperature:.2f}°C hors plage pour {normalized_probe}")
198             return None
199
200         # Affichage de confirmation de conversion réussie
201         print(f"[CONV] Conversion réussie: {temperature:.2f}°C -> {resistance:.2f} ohms")
202         # Retour de la résistance calculée
203         return resistance
204
205     except ValueError as e:
206         # Affichage d'erreur lors de la conversion principale
207         print(f"[CONV] Erreur de conversion: {e}")
208
209         # Liste des noms alternatifs pour tentatives de récupération
210         alternative_names = [
211             "PT1000",
212             "Ni1000 TK5000",
213             "Ni1000 TK6180",
214             "NTC 10k 3977",
215             "NTC 1k 3528",
216             "De Dietrich AF60",
217             "Siemens QAC32"
218         ]
219
220         # Tentative de correspondance avec les noms alternatifs
221         for alt_name in alternative_names:
222             # Comparaison de noms avec normalisation (minuscules, espaces)
223             if probe_type.lower().replace("_", " ") in alt_name.lower():
224                 try:
225                     # Tentative de conversion avec le nom alternatif
226                     resistance = temperature_to_resistance_dynamic(temperature, alt_name)
227                     # Vérification de la validité du résultat
228                     if resistance is not None:
229                         # Affichage de succès avec nom alternatif utilisé
230                         print(f"[CONV] Conversion avec nom alternatif '{alt_name}': {temperature:.2f}°C -> {resistance:.2f} ohms")
231                         return resistance
232                 except ValueError:
233                     # Continuation de la boucle si échec avec ce nom alternatif
234                     continue
235
236         # Affichage d'erreur si aucune sonde compatible trouvée
237         print(f"[CONV] Aucune sonde compatible trouvée pour '{probe_type}'")
238         # Affichage de la liste des sondes disponibles pour assistance
239         print(f"[CONV] Sondes disponibles: {list(SENSOR_TABLES.keys())}")
240         return None
241
242 # -----
243 # Fonctions utilitaires et validation
244 # -----
245
246 def get_supported_probe_types():
247     """
248     @brief  Retourne la liste des types de sondes supportés.
249     @details Fonction utilitaire qui extrait la liste complète des noms
250             de capteurs disponibles dans les tables de conversion.
251
252     @return Liste des noms de capteurs disponibles pour conversion.
253     """
254     # Retour direct des clés du dictionnaire des tables de capteurs
255     return list(SENSOR_TABLES.keys())
256
257 def validate_temperature_range(temperature, probe_type):
258     """

```

```
259 @brief Valide si une température est dans la plage supportée par une sonde.
260 @details Fonction de validation qui vérifie si une température donnée
261 est compatible avec la plage de fonctionnement d'un type de
262 capteur spécifique selon sa table de calibration.
263
264 @param temperature Température à valider [°C].
265 @param probe_type Type de sonde à vérifier.
266
267 @return True si la température est dans la plage supportée.
268 """
269 try:
270     # Dictionnaire de normalisation des noms pour la validation
271     probe_mapping = {
272         "PT1000": "PT1000",
273         "Ni1000_TK5000": "Ni1000 TK5000",
274         "NTC_10k": "NTC 10k 3977"
275     }
276     # Application de la normalisation du nom de sonde
277     normalized_probe = probe_mapping.get(probe_type, probe_type)
278
279     # Vérification de l'existence de la sonde dans les tables
280     if normalized_probe not in SENSOR_TABLES:
281         return False
282
283     # Récupération de la table de conversion pour la sonde
284     table = SENSOR_TABLES[normalized_probe]
285     # Extraction de la température minimale de la table
286     t_min, _ = table[0]
287     # Extraction de la température maximale de la table
288     t_max, _ = table[-1]
289
290     # Vérification si la température est dans la plage [t_min, t_max]
291     return t_min <= temperature <= t_max
292
293 except (KeyError, IndexError):
294     # Retour de False en cas d'erreur d'accès aux données
295     return False
```